



ESCUELA POLITÉCNICA NACIONAL
ESCUELA DE FORMACIÓN DE TECNÓLOGOS



PROGRAMACION ORIENTADA A OBJETOS

ASIGNATURA:

Programación orientada a objetos

PROFESOR:

Ing. Yadira Gissela Franco Rocha

PERÍODO ACADÉMICO:

2026-A

Proyecto Final

TÍTULO:

**Sistema de Gestión Académica para Estudiantes y
Profesores**

EduControl

Integrantes:

Alessandro Pillajo

Alex Lascano

Fecha de entrega: 22/07/2026

1.- Índice

1. Introducción
2. Propósito de la práctica
3. Objetivo general
4. Objetivos específicos
5. Desarrollo
 - 5.1 Temática del proyecto
 - 5.2 Arquitectura del sistema
 - 5.3 Programación Orientada a Objetos
 - 5.4 Explicación de los paquetes
 - 5.5 Explicación de las clases principales
 - 5.6 Diagrama UML
 - 5.7 Base de datos
 - 5.8 Inicio de sesión
 - 5.9 Roles del sistema
 - 5.10 Módulo CRUD
 - 5.11 Reportes
 - 5.12 Validaciones
6. Capturas del sistema
7. Conclusiones
8. Recomendaciones
9. Bibliografía
10. Anexos

1. Introducción

La Programación Orientada a Objetos constituye uno de los paradigmas más utilizados en el desarrollo de software moderno debido a que facilita la reutilización del código, la organización de los componentes y el mantenimiento de las aplicaciones. En este contexto, el presente proyecto consiste en el desarrollo de un sistema de escritorio utilizando JavaFX como interfaz gráfica y MySQL como gestor de base de datos, aplicando los principios fundamentales de la Programación Orientada a Objetos.

El sistema desarrollado tiene como finalidad administrar la información académica de estudiantes y profesores mediante una interfaz intuitiva que permita realizar operaciones de registro, consulta, modificación y eliminación de datos. Además, incorpora un sistema de autenticación basado en roles, lo que garantiza que cada usuario tenga acceso únicamente a las funciones correspondientes a su perfil.

Durante el desarrollo del proyecto se implementó una arquitectura organizada por paquetes, permitiendo separar las responsabilidades entre la lógica del negocio, el acceso a datos, los controladores de la interfaz y los modelos de información. Asimismo, se utilizaron clases, herencia, encapsulamiento, interfaces y otros conceptos propios de la Programación Orientada a Objetos, favoreciendo una aplicación modular, escalable y fácil de mantener.

Finalmente, el sistema integra una base de datos relacional que almacena la información de usuarios, estudiantes, profesores, asignaciones y reportes, permitiendo que toda la información permanezca persistente y pueda ser consultada en cualquier momento.

2. Propósito de la práctica

El propósito del presente proyecto es desarrollar una aplicación de escritorio que permita gestionar la información académica de una institución educativa mediante el uso de JavaFX y una base de datos MySQL, aplicando los principios fundamentales de la Programación Orientada a Objetos.

Con este proyecto se busca fortalecer los conocimientos adquiridos durante la asignatura, implementando una solución informática organizada por capas y paquetes, capaz de administrar usuarios, estudiantes, profesores, asignaciones y reportes, utilizando operaciones CRUD conectadas directamente a una base de datos.

Además, el desarrollo del sistema permite comprender la importancia del uso de patrones de diseño, interfaces, encapsulamiento, herencia y polimorfismo para construir aplicaciones robustas, reutilizables y fáciles de mantener.

3. Objetivo General

Desarrollar una aplicación de escritorio utilizando JavaFX y MySQL que permita administrar la información académica de estudiantes y profesores mediante un sistema de autenticación por roles, implementando operaciones CRUD y aplicando los principios de la Programación Orientada a Objetos.

4. Objetivos Específicos

- Diseñar una aplicación organizada mediante paquetes que facilite la separación entre la interfaz gráfica, la lógica del negocio y el acceso a los datos.
- Implementar un sistema de autenticación que permita controlar el acceso de los usuarios según el rol asignado.
- Desarrollar operaciones CRUD para la administración de estudiantes, profesores, asignaciones y reportes utilizando una base de datos MySQL.
- Aplicar los principios de encapsulamiento, herencia, abstracción y polimorfismo durante el desarrollo del sistema.
- Implementar validaciones que garanticen la integridad de la información ingresada por el usuario.

5. DESARROLLO

5.1 Temática del Proyecto

El proyecto desarrollado lleva por nombre **EduControl: Sistema de Gestión Académica**, una aplicación de escritorio creada con JavaFX y MySQL cuyo objetivo es facilitar la administración de la información académica de una institución educativa.

El sistema permite gestionar usuarios, estudiantes, profesores, asignaciones y reportes mediante una interfaz gráfica intuitiva, organizada y de fácil utilización. Para garantizar la seguridad de la información, la aplicación incorpora un sistema de autenticación que valida las credenciales del usuario antes de permitir el acceso al sistema.

Cada usuario posee un rol específico que determina las funcionalidades disponibles dentro de la aplicación, permitiendo administrar la información de manera ordenada y segura.

La persistencia de los datos se realiza mediante una base de datos MySQL, donde se almacenan todos los registros del sistema, evitando la pérdida de información y permitiendo realizar consultas, modificaciones y eliminaciones de manera eficiente.

5.2 Arquitectura del Sistema

El proyecto fue desarrollado siguiendo una arquitectura por capas, la cual permite dividir la aplicación en diferentes módulos según su responsabilidad. Esta organización facilita el mantenimiento del código, mejora su reutilización y hace que futuras ampliaciones sean más sencillas.

La estructura principal del proyecto se encuentra organizada de la siguiente manera:

Proyecto_POO/

```
|
|
|— src/
|   |
|   |— main/
|   |   |
|   |   |— java/
|   |   |   |
|   |   |   |— com/
|   |   |   |   |
|   |   |   |   |— proyecto-poo/
|   |   |   |   |   |
|   |   |   |   |   |— app/
|   |   |   |   |   |   |
|   |   |   |   |   |   |— MainApp.java
|   |   |   |   |   |
|   |   |   |   |   |— controller/
|   |   |   |   |   |   |
|   |   |   |   |   |   |— LoginController.java
|   |   |   |   |   |   |— DashboardController.java
|   |   |   |   |   |   |— UsuarioController.java
|   |   |   |   |   |   |— EstudianteController.java
|   |   |   |   |   |   |— ProfesorController.java
|   |   |   |   |   |   |— AsignacionController.java
|   |   |   |   |   |   |— ReporteController.java
|   |   |   |   |   |   |— AdminEstudianteController.java
|   |   |   |   |   |
|   |   |   |   |   |— dao/
|   |   |   |   |   |   |
|   |   |   |   |   |   |— ICRUD.java
|   |   |   |   |   |   |— UsuarioDAO.java
|   |   |   |   |   |   |— EstudianteDAO.java
|   |   |   |   |   |   |— ProfesorDAO.java
|   |   |   |   |   |   |— AsignacionDAO.java
|   |   |   |   |   |   |— ReporteDAO.java
```

```

├── AdminEstudianteDAO.java
│   ├── db/
│   │   └── Conexion.java
│   ├── model/
│   │   ├── Persona.java
│   │   ├── Usuario.java
│   │   ├── Estudiante.java
│   │   ├── Profesor.java
│   │   ├── Asignacion.java
│   │   └── Reporte.java
│   └── util/
│       └── Sesion.java
├── resources/
│   ├── css/
│   │   └── estilos.css
│   ├── view/
│   │   ├── login.fxml
│   │   ├── dashboard.fxml
│   │   ├── usuarios.fxml
│   │   ├── estudiantes.fxml
│   │   ├── profesores.fxml
│   │   ├── asignaciones.fxml
│   │   ├── reportes.fxml
│   │   └── adminEstudiantes.fxml |
├── database/
│   └── script_bd_proyecto_poo.sql
├── dist/
└── ProyectoFinal-POO.jar
```

```
|   └── ProyectoFinal-POO.exe
|
|── docs/
|   ├── InformeProyecto.pdf
|   ├── ManualUsuario.pdf
|   └── UML_EduControl.png
|
|── video/
|   └── DemoProyectoPOO.mp4
|
|── README.md
|── pom.xml
|── .gitignore
└── LICENSE
```

Cada paquete cumple una función específica dentro del sistema, evitando que toda la lógica de la aplicación se encuentre concentrada en una sola clase.

Esta separación mejora la legibilidad del código y facilita que varios desarrolladores puedan trabajar simultáneamente sobre el proyecto.

5.3 Aplicación de los Principios de la Programación Orientada a Objetos

Durante el desarrollo de **EduControl** se implementaron los cuatro pilares fundamentales de la Programación Orientada a Objetos.

Encapsulamiento

El encapsulamiento consiste en proteger los atributos de una clase evitando que sean modificados directamente desde otras clases.

En el proyecto, las clases del paquete **model** poseen atributos privados y utilizan métodos *getter* y *setter* para acceder a ellos.

Por ejemplo, la clase **Usuario** almacena información como el nombre de usuario, contraseña y rol mediante atributos privados, garantizando un acceso controlado.

Este principio mejora la seguridad del código y evita modificaciones incorrectas de los datos.

Herencia

La herencia permite reutilizar características comunes entre varias clases.

En el proyecto existe una clase denominada **Persona**, la cual contiene información común como nombres, apellidos, identificación y demás datos personales.

A partir de esta clase se derivan las clases:

- Estudiante
- Profesor

Gracias a la herencia se evita duplicar código, ya que ambas clases reutilizan los atributos y métodos definidos en **Persona**, agregando únicamente las características propias de cada tipo de usuario.

Abstracción

La abstracción consiste en representar únicamente las características esenciales de un objeto.

En este proyecto cada clase representa una entidad real del sistema académico.

Por ejemplo:

- Usuario representa a las personas que pueden iniciar sesión.
- Estudiante representa la información académica de un alumno.
- Profesor representa al personal docente.
- Asignación representa la relación entre profesores y estudiantes.
- Reporte representa la información generada por el sistema.

Cada clase contiene únicamente los atributos y métodos necesarios para cumplir su función.

Polimorfismo

El polimorfismo permite que diferentes objetos puedan utilizar una misma interfaz o método de formas distintas.

En el proyecto este principio se evidencia mediante la interfaz **ICRUD**, la cual define las operaciones básicas:

- Insertar
- Actualizar
- Eliminar
- Buscar

Las diferentes clases DAO implementan esta interfaz adaptando el comportamiento de cada operación según la entidad que administran.

Gracias a ello se mantiene una estructura uniforme para todas las operaciones CRUD del sistema.

5.4 Explicación de los Paquetes

Con el objetivo de mantener una arquitectura organizada y facilitar el mantenimiento del software, el proyecto fue dividido en varios paquetes. Cada uno posee responsabilidades claramente definidas.

Paquete app

Este paquete contiene la clase principal de la aplicación.

Dentro de él se encuentra la clase **MainApp**, encargada de iniciar la ejecución del sistema JavaFX.

Entre sus principales responsabilidades se encuentran:

- Inicializar JavaFX.
- Cargar la ventana principal del sistema.
- Abrir el archivo **login.fxml**.
- Configurar el escenario principal (*Stage*).
- Iniciar la interacción entre el usuario y la aplicación.

Este paquete representa el punto de entrada del programa.

Paquete controller

El paquete **controller** contiene todas las clases encargadas de controlar la interacción entre la interfaz gráfica y la lógica del sistema.

Cada archivo FXML posee un controlador asociado.

En el proyecto se encuentran los siguientes controladores:

- **LoginController**
- **DashboardController**
- **EstudianteController**
- **ProfesorController**
- **AsignacionController**
- **ReporteController**
- **AdminEstudianteController**

Cada uno administra los eventos generados por los botones, tablas, formularios y demás componentes gráficos, coordinando las operaciones con las clases DAO.

Paquete dao

El paquete DAO (Data Access Object) es el encargado de la comunicación entre la aplicación y la base de datos MySQL.

Aquí se implementan todas las operaciones CRUD del sistema.

Las principales clases son:

- UsuarioDAO
- EstudianteDAO
- ProfesorDAO
- AsignacionDAO
- ReporteDAO
- AdminEstudianteDAO

Estas clases contienen las consultas SQL necesarias para insertar, actualizar, eliminar y consultar información.

Además, todas implementan la interfaz **ICRUD**, garantizando una estructura homogénea para el acceso a los datos.

Paquete db

Este paquete contiene la clase responsable de establecer la conexión con la base de datos MySQL.

Su función principal es:

- Abrir la conexión mediante JDBC.
- Validar que la conexión sea exitosa.
- Proporcionar la conexión a las clases DAO.
- Gestionar posibles errores durante la conexión.

Este paquete constituye el puente entre la aplicación y el servidor de base de datos.

Paquete model

El paquete **model** representa las entidades del sistema.

Cada clase modela un objeto del mundo real.

Entre ellas se encuentran:

- Persona
- Usuario
- Estudiante
- Profesor

- Asignacion
- Reporte

Estas clases almacenan la información correspondiente a cada entidad mediante atributos privados y métodos de acceso (*getters* y *setters*), respetando el principio de encapsulamiento.

Paquete util

El paquete **util** contiene clases auxiliares utilizadas por diferentes módulos del sistema.

En este proyecto destaca la clase **Sesion**, cuya función es almacenar temporalmente la información del usuario autenticado durante la ejecución de la aplicación.

Esto permite conocer qué usuario ha iniciado sesión y mantener dicha información disponible mientras la aplicación permanece abierta.

Recursos de la aplicación

Dentro de la carpeta **resources** se encuentran todos los archivos necesarios para la interfaz gráfica.

- **Carpeta view**

Contiene las interfaces desarrolladas con JavaFX mediante archivos FXML:

- login.fxml
- dashboard.fxml
- estudiantes.fxml
- profesores.fxml
- asignacion.fxml
- reportes.fxml
- AdminEstudiantes.fxml

Cada uno define visualmente las ventanas que utiliza el sistema.

Carpeta css

Contiene el archivo:

estilos.css

Este archivo permite personalizar la apariencia del sistema, incluyendo colores, botones, fuentes, tablas y otros componentes gráficos, proporcionando una interfaz más agradable y uniforme.

5.5 Explicación de las Clases Principales

El sistema **EduControl** fue desarrollado mediante múltiples clases, donde cada una cumple una responsabilidad específica. Esta separación de responsabilidades facilita el mantenimiento del software y permite reutilizar código.

Clase **MainApp**

La clase **MainApp** constituye el punto de inicio de la aplicación.

Su función principal consiste en iniciar JavaFX y cargar la primera ventana que visualizará el usuario, correspondiente al inicio de sesión.

Las principales responsabilidades de esta clase son:

- Inicializar la aplicación JavaFX.
- Crear el escenario principal (*Stage*).
- Cargar el archivo **login.fxml**.
- Mostrar la ventana principal.
- Finalizar correctamente la aplicación cuando el usuario la cierre.

Gracias a esta clase toda la aplicación comienza su ejecución de forma organizada.

Clase **Conexion**

La clase **Conexion** pertenece al paquete **db**.

Es una de las clases más importantes del sistema debido a que permite establecer la comunicación con la base de datos MySQL utilizando JDBC.

Entre sus funciones principales se encuentran:

- Abrir la conexión con MySQL.
- Validar que la conexión se haya realizado correctamente.
- Proporcionar una instancia de conexión a los DAO.
- Cerrar la conexión cuando sea necesario.
- Manejar excepciones relacionadas con la base de datos.

Al centralizar la conexión en una única clase se evita repetir código en todo el proyecto.

Clase **Usuario**

La clase **Usuario** representa las credenciales utilizadas para ingresar al sistema.

Entre los atributos principales se encuentran:

- ID
- Nombre de usuario

- Contraseña
- Rol

Cada atributo es privado y únicamente puede ser modificado mediante métodos **Getter** y **Setter**, cumpliendo con el principio de encapsulamiento.

Los objetos de esta clase son utilizados principalmente durante el proceso de autenticación.

Clase Persona

La clase **Persona** representa la información general de una persona.

Esta clase fue creada para evitar duplicar código entre estudiantes y profesores.

Entre los atributos comunes se encuentran:

- Identificación
- Nombres
- Apellidos
- Dirección
- Teléfono
- Correo electrónico

Gracias a esta clase es posible reutilizar información común mediante el mecanismo de herencia.

Clase Estudiante

La clase **Estudiante** hereda de **Persona**.

Además de los datos personales incorpora información académica propia de cada estudiante.

Entre sus principales responsabilidades están:

- Representar la información del estudiante.
- Almacenar datos académicos.
- Facilitar el registro dentro de la base de datos.
- Permitir consultas y modificaciones.

Cada objeto de esta clase representa un estudiante registrado en el sistema.

Clase Profesor

La clase **Profesor** también hereda de **Persona**.

Su finalidad es almacenar la información correspondiente al personal docente.

Entre sus atributos pueden encontrarse:

- Especialidad
- Área académica

- Horario
- Asignaturas

Esta clase permite administrar la información de los profesores mediante operaciones CRUD.

Clase Asignacion

La clase **Asignacion** representa la relación existente entre profesores y estudiantes.

Permite registrar la asignación de materias, cursos o responsabilidades académicas.

Gracias a esta clase es posible mantener organizada la distribución académica dentro del sistema.

Clase Reporte

La clase **Reporte** representa la información generada por el sistema.

Su finalidad consiste en organizar los datos obtenidos desde la base de datos para mostrarlos posteriormente al usuario mediante tablas o consultas.

Clases DAO

Cada entidad del sistema posee una clase DAO encargada del acceso a la base de datos.

Entre ellas se encuentran:

- UsuarioDAO
- EstudianteDAO
- ProfesorDAO
- AsignacionDAO
- ReporteDAO

Todas estas clases realizan operaciones SQL utilizando PreparedStatement para evitar errores y mejorar la seguridad del sistema.

Las principales operaciones implementadas son:

- INSERT
- UPDATE
- DELETE
- SELECT

Cada DAO trabaja únicamente con la tabla correspondiente, respetando el principio de responsabilidad única.

Interfaces

El proyecto utiliza la interfaz **ICRUD**, la cual define las operaciones básicas que deben implementar todas las clases DAO.

Entre los métodos declarados se encuentran:

- insertar()
- actualizar()
- eliminar()
- buscar()

La utilización de interfaces favorece el polimorfismo y facilita futuras ampliaciones del sistema.

Sesion

La clase **Sesion**, ubicada dentro del paquete **util**, almacena temporalmente la información del usuario autenticado.

Gracias a esta clase el sistema puede conocer durante toda la ejecución:

- Usuario conectado.
- Nombre del usuario.
- Rol.
- Permisos disponibles.

Esta información permite controlar el acceso a cada módulo.

5.6 Diagrama UML

Para representar la estructura del sistema se elaboró un diagrama UML de clases que muestra las relaciones entre las entidades principales.

El diagrama evidencia el uso de la Programación Orientada a Objetos mediante herencia, asociación e implementación de interfaces.

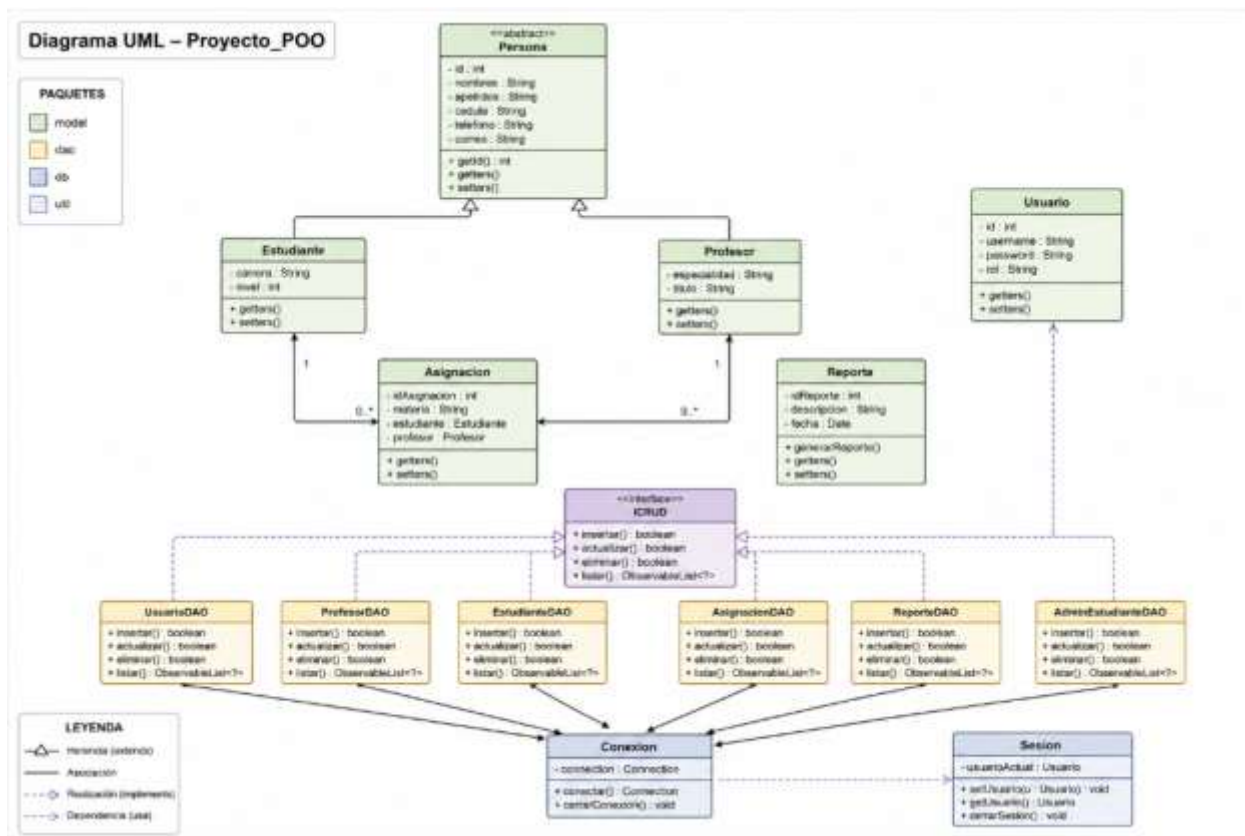


Figura 1. Diagrama UML del sistema EduControl.

Descripción de la figura:

En el diagrama se observa que la clase **Persona** actúa como clase base de **Estudiante** y **Profesor**, mientras que las clases DAO implementan la interfaz **ICRUD** para realizar las operaciones CRUD. Además, la clase **Conexion** proporciona acceso a la base de datos y los controladores coordinan la interacción entre la interfaz gráfica y la lógica del sistema.

5.7 Base de Datos

El sistema utiliza **MySQL** como gestor de base de datos debido a su estabilidad, facilidad de integración con Java y alto rendimiento.

La comunicación entre JavaFX y MySQL se realiza mediante el controlador JDBC.

La base de datos almacena toda la información utilizada por el sistema, evitando que los datos se pierdan al cerrar la aplicación.

Entre las tablas principales se encuentran:

- usuarios
- estudiantes
- profesores
- asignaciones
- reportes

Cada tabla posee una clave primaria que identifica de manera única cada registro.

Las relaciones entre las tablas permiten mantener la integridad de los datos y evitar inconsistencias.

El acceso a la base de datos se realiza exclusivamente mediante las clases DAO.

Script SQL

El script de creación de la base de datos se entrega como un archivo independiente denominado **script_bd_proyecto_poo.sql**, ubicado dentro de la carpeta **database** del proyecto. Este archivo contiene la creación de la base de datos, tablas, claves primarias, claves foráneas, relaciones y registros de prueba necesarios para el funcionamiento del sistema.

Captura de la Base de Datos

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and search. The main editor displays the following SQL script:

```

1 • CREATE DATABASE proyecto_poo;
2
3 • USE proyecto_poo;
4
5 • CREATE TABLE usuarios (
6     id INT PRIMARY KEY AUTO_INCREMENT,
7     nombre VARCHAR(100) NOT NULL,
8     apellido VARCHAR(100) NOT NULL,
9     correo VARCHAR(150) UNIQUE NOT NULL,
10    usuario VARCHAR(50) UNIQUE NOT NULL,
11    contraseña VARCHAR(255) NOT NULL,
12    rol VARCHAR(20) NOT NULL -- 'ESTUDIANTE' o 'PROFESOR'
13 );
14
15 • CREATE TABLE estudiantes (
16     id INT PRIMARY KEY,
17     cedula VARCHAR(20) UNIQUE NOT NULL,
18     carrera VARCHAR(100),
19     nivel INT,
20     FOREIGN KEY (id) REFERENCES usuarios(id) ON DELETE CASCADE
21 );
22
23 • CREATE TABLE profesores (

```

Below the script, the 'Result Grid' is visible, showing a table with columns: id, titulo, descripcion, fecha, usuario_id, and profesor_id. The first row contains sample data, and the second row shows NULL values.

	id	titulo	descripcion	fecha	usuario_id	profesor_id
▶	1	Informe inicial	Avance de proyecto	2026-07-21	1	2
*	NULL	NULL	NULL	NULL	NULL	NULL

Figura 2. Base de datos del sistema EduControl.

5.8 Inicio de Sesión

El sistema **EduControl** incorpora un módulo de autenticación que permite verificar la identidad del usuario antes de conceder acceso a la aplicación.

El proceso de autenticación comienza cuando el usuario ingresa su nombre de usuario y contraseña en la ventana principal del sistema. Posteriormente, el controlador del login envía esta información a la clase **UsuarioDAO**, la cual consulta la base de datos para comprobar si las credenciales son válidas.

Como medida de seguridad, las contraseñas almacenadas en la base de datos se encuentran cifradas mediante **BCrypt**, evitando que puedan visualizarse en texto plano. Durante el inicio de sesión, la contraseña ingresada por el usuario se compara con el hash almacenado utilizando el método de verificación correspondiente.

Si las credenciales son correctas, el sistema identifica el rol del usuario y carga automáticamente la interfaz correspondiente. En caso contrario, se muestra un mensaje indicando que el usuario o la contraseña son incorrectos.

Este mecanismo garantiza que únicamente los usuarios autorizados puedan acceder al sistema y protege la información almacenada.



Figura 3. Pantalla de inicio de sesión.

5.9 Control de Acceso por Roles

Uno de los aspectos más importantes del sistema es el control de acceso basado en roles.

Cada usuario registrado posee un rol determinado que define las funcionalidades disponibles dentro de la aplicación. Esto permite restringir el acceso a módulos sensibles y mejorar la seguridad del sistema.

Los roles implementados son:

Administrador

El Administrador posee acceso completo a la aplicación.

Entre sus funciones principales se encuentran:

- Gestionar usuarios.
- Registrar estudiantes.
- Registrar profesores.
- Administrar asignaciones.
- Consultar reportes.
- Editar información.
- Eliminar registros.
- Configurar el sistema.

Es el usuario con mayor nivel de privilegios.

Profesor

El Profesor puede acceder únicamente a las funciones relacionadas con la gestión académica que le corresponden.

Entre ellas:

- Consultar estudiantes asignados.
- Visualizar información académica.
- Registrar información autorizada.
- Consultar reportes.

No tiene permisos para administrar usuarios ni modificar configuraciones generales.

Estudiante

El Estudiante posee un acceso restringido orientado únicamente a la consulta de información.

Puede realizar acciones como:

- Consultar datos personales.

- Visualizar asignaciones.
- Revisar información académica.

No puede modificar registros de otros usuarios.

Este esquema garantiza que cada usuario tenga acceso únicamente a las funciones necesarias para desempeñar su actividad.

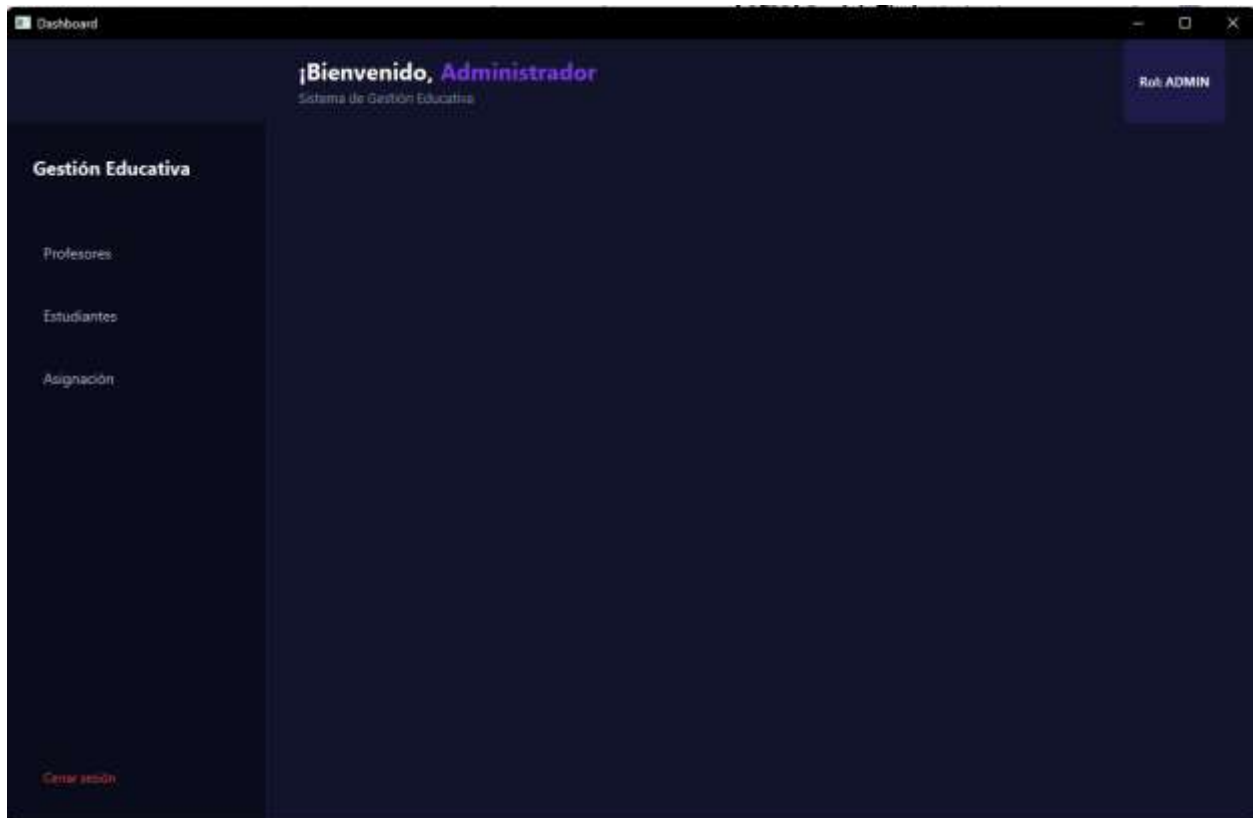


Figura 4. Dashboard según el rol del usuario.

5.10 Módulo CRUD

El sistema implementa las cuatro operaciones fundamentales para la administración de la información:

- **Crear (Create)**

Permite registrar nuevos datos en la base de datos.

Durante esta operación el sistema valida que la información ingresada sea correcta antes de almacenarla.

Al finalizar el proceso se muestra un mensaje indicando que el registro fue creado exitosamente.

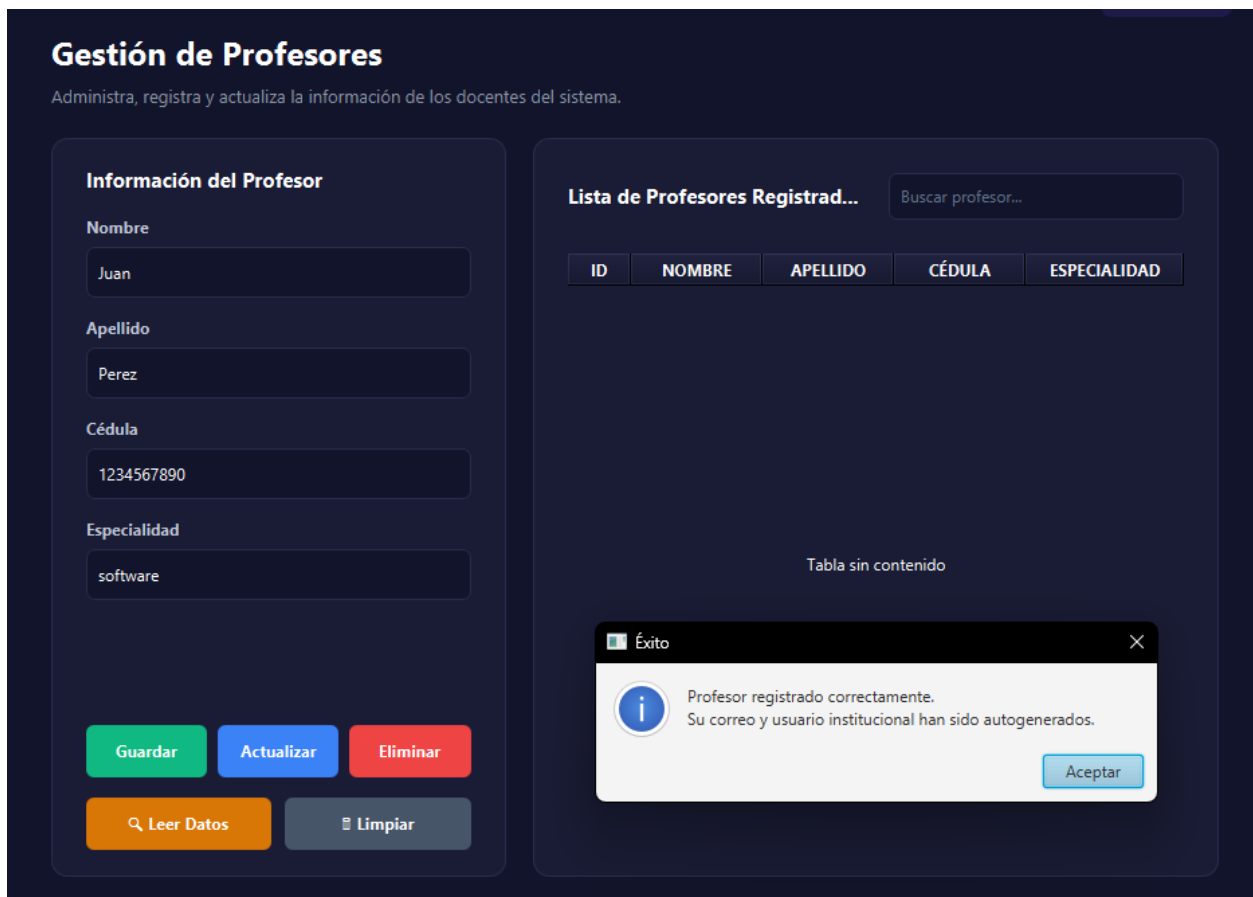


Figura 5. Registro de información.

Consultar (Read)

La consulta permite visualizar la información almacenada mediante tablas JavaFX (*TableView*). Los datos son obtenidos directamente desde MySQL utilizando consultas SQL implementadas en las clases DAO.

Cada registro aparece organizado en filas y columnas para facilitar su lectura.

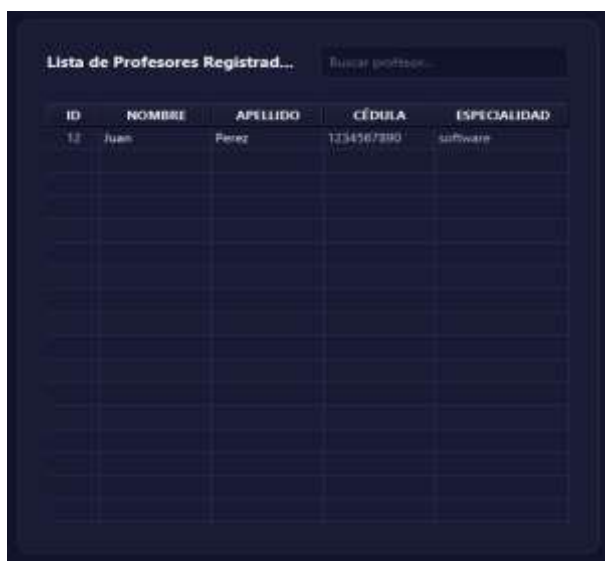


Figura 6. Consulta de registros.

Actualizar (Update)

Esta operación permite modificar la información previamente registrada.

Cuando el usuario selecciona un registro, los datos son cargados automáticamente en el formulario correspondiente para que puedan ser editados.

Posteriormente, la información actualizada se guarda nuevamente en la base de datos.

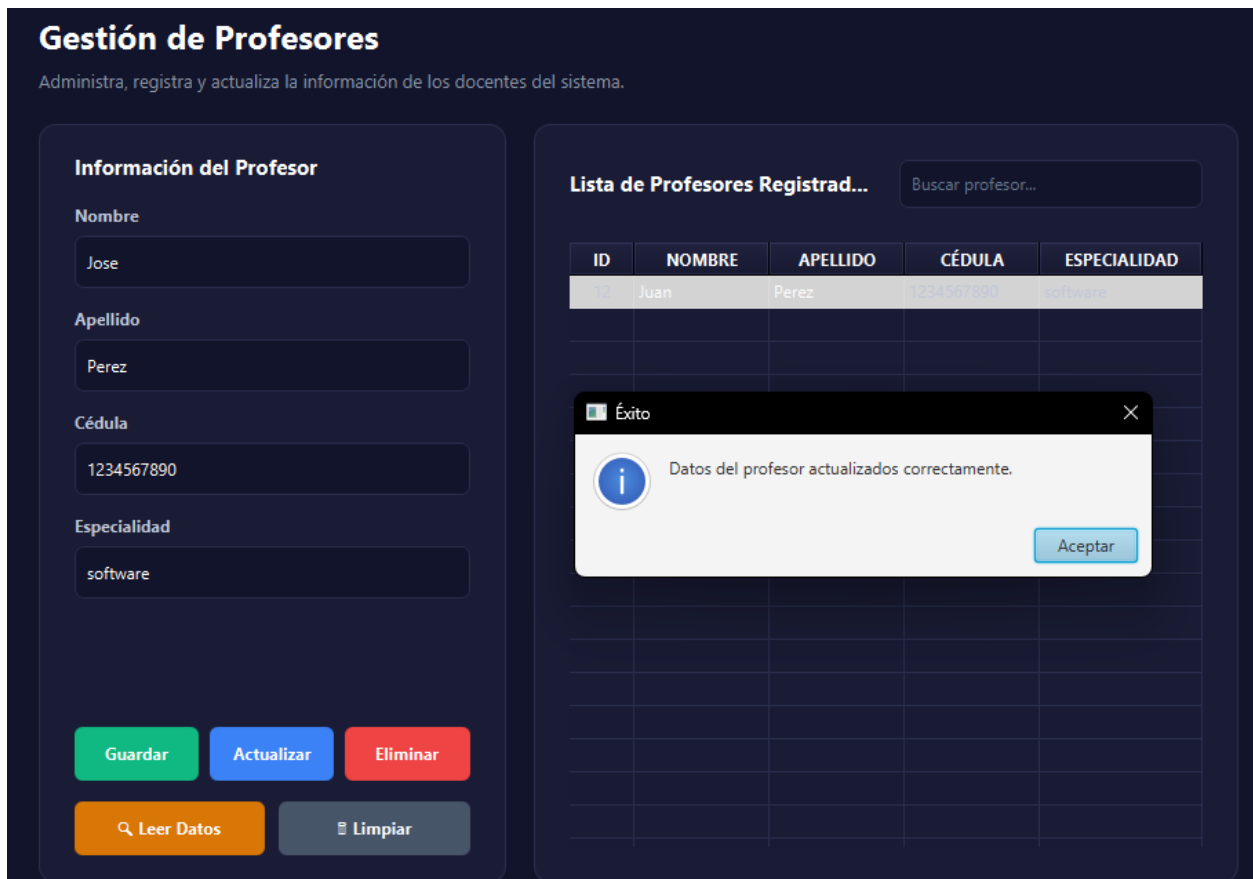


Figura 7. Actualización de registros.

Eliminar (Delete)

Antes de eliminar un registro, el sistema solicita una confirmación al usuario mediante una ventana de diálogo.

Esta validación evita la eliminación accidental de información.

Una vez confirmada la operación, el registro es eliminado de la base de datos y la tabla se actualiza automáticamente.

The screenshot displays a web application titled "Gestión de Profesores" with the subtitle "Administra, registra y actualiza la información de los docentes del sistema." The interface is divided into two main sections. On the left, the "Información del Profesor" form contains input fields for "Nombre" (filled with "Jose"), "Apellido" (filled with "Perez"), "Cédula" (filled with "1234567890"), and "Especialidad" (filled with "software"). Below these fields are four buttons: "Guardar" (green), "Actualizar" (blue), "Eliminar" (red), and "Leer Datos" (orange). On the right, the "Lista de Profesores Registrados" section features a search bar and a table with columns "ID", "NOMBRE", "APELLIDO", "CÉDULA", and "ESPECIALIDAD". The first row of the table shows "12", "Jose", "Perez", "1234567890", and "software". A modal dialog titled "Confirmar Eliminación" is overlaid on the table, asking "¿Está seguro de eliminar a este profesor? Se borrarán sus credenciales." with "Sí" and "No" buttons.

Figura 8. Eliminación de registros.

5.11 Validaciones del Sistema

Para garantizar la calidad de la información almacenada, el sistema incorpora diversas validaciones antes de ejecutar cualquier operación.

Entre las principales validaciones implementadas se encuentran:

- Verificación de campos obligatorios.
- Validación de credenciales de inicio de sesión.
- Restricción de números negativos en campos numéricos.
- Validación de formatos de correo electrónico.
- Prevención de registros duplicados.
- Confirmación antes de eliminar información.
- Manejo de errores de conexión con la base de datos.
- Mensajes de éxito o error después de cada operación.

Estas validaciones permiten reducir errores de ingreso de datos y mejorar la experiencia del usuario.

The screenshot displays a web application interface with a dark theme. On the left, there is a form titled "Información del Profesor" with fields for "Nombre" (containing "Pepe"), "Apellido" (placeholder "Ingrese el apellido"), "Cédula" (containing "1515151515"), and "Especialidad" (containing "redes"). Below the form are buttons for "Guardar" (green), "Actualizar" (blue), "Eliminar" (red), "Leer Datos" (orange with a magnifying glass icon), and "Limpiar" (grey with a trash icon). On the right, there is a section titled "Lista de Profesores Registrad..." with a search bar "Buscar profesor...". Below this is a table with headers "ID", "NOMBRE", "APELLIDO", "CÉDULA", and "ESPECIALIDAD". The table is currently empty, showing "Tabla sin contenido". A modal dialog titled "Campos Incompletos" is open, displaying a warning icon and the message "Debe rellenar todos los campos del formulario." with an "Aceptar" button.

Figura 9. Mensajes de validación del sistema.

6. Capturas del Sistema

En esta sección se presentan las principales pantallas de la aplicación desarrollada.



Figura 10. Pantalla de inicio de sesión

Descripción: Interfaz principal del sistema donde el usuario ingresa sus credenciales para autenticarse. Una vez verificadas, el sistema permite el acceso a las funcionalidades correspondientes según el rol asignado.

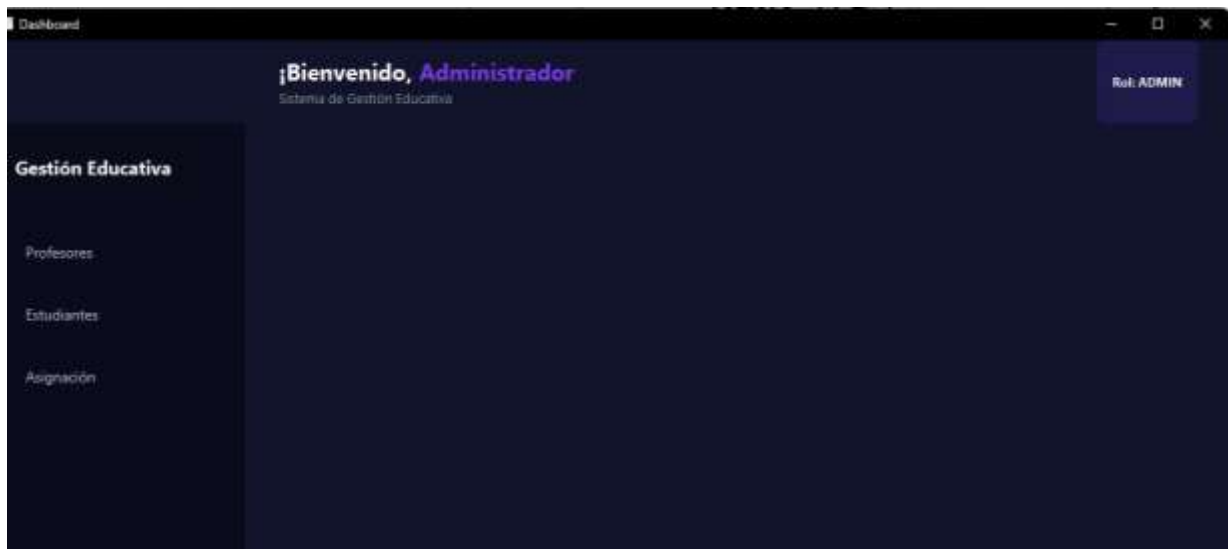


Figura 11. Menú principal

Descripción: Pantalla principal del sistema que muestra las opciones disponibles para el usuario autenticado, permitiendo acceder a los diferentes módulos de la aplicación.

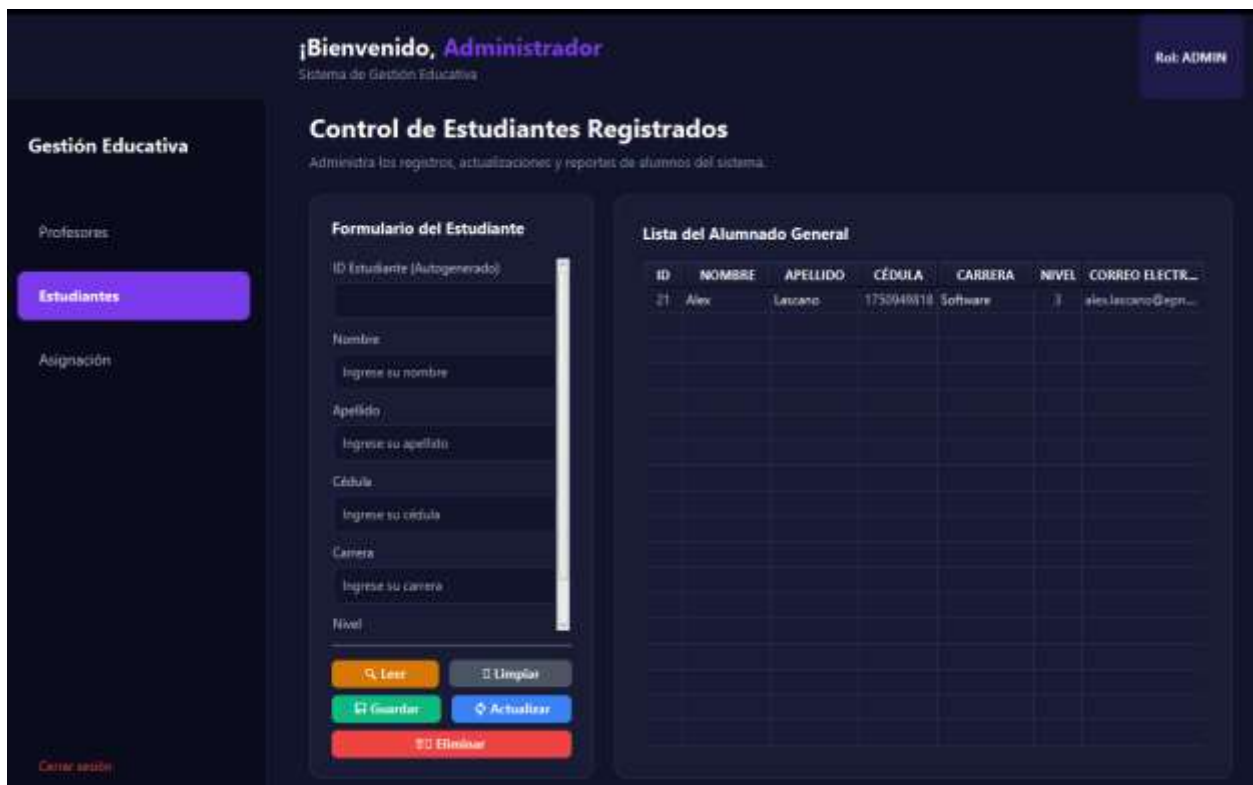


Figura 12. Gestión de estudiantes

Descripción: Módulo destinado a la administración de estudiantes, permitiendo realizar las operaciones de registro, consulta, actualización y eliminación de información.

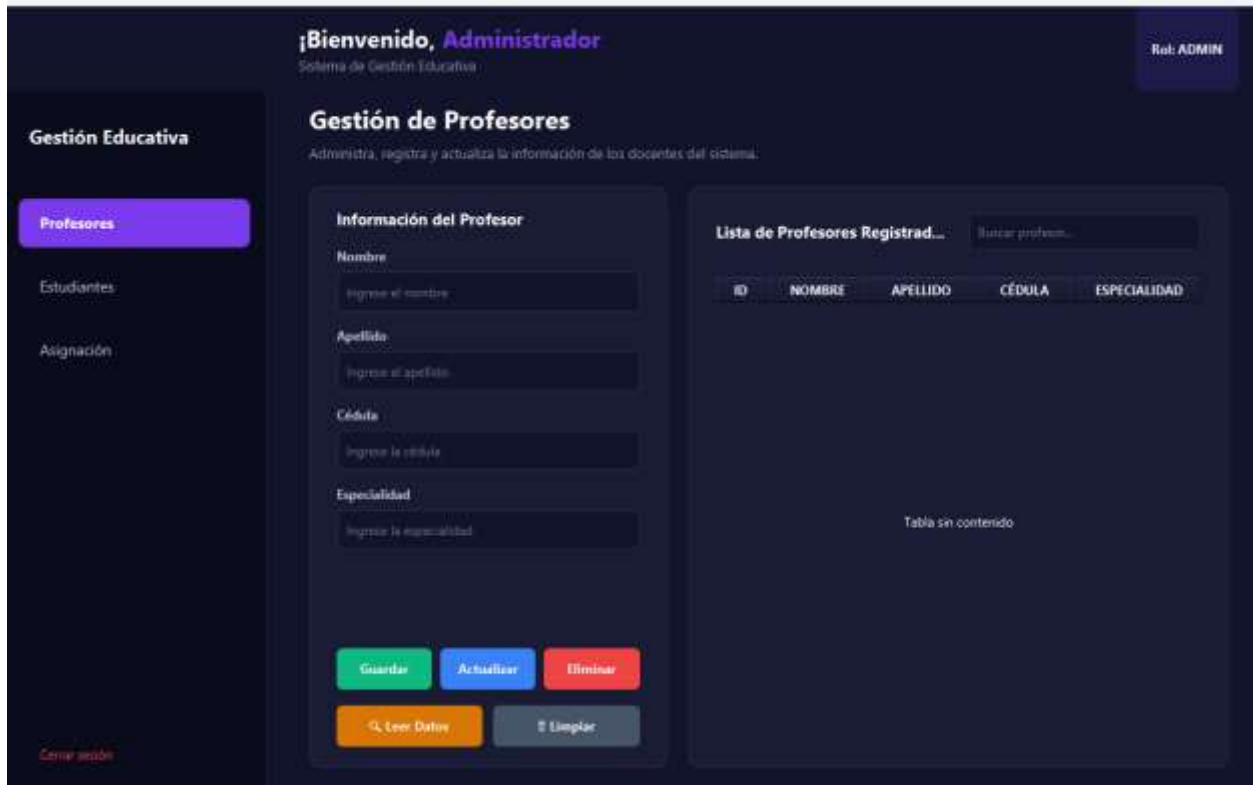


Figura 13. Gestión de profesores

Descripción: Interfaz encargada de administrar la información de los profesores mediante operaciones CRUD, facilitando el mantenimiento de los datos registrados.



Figura 14. Gestión de asignaciones

Descripción: Módulo que permite gestionar las asignaciones académicas entre estudiantes y profesores, organizando la información de manera eficiente.

Gestión de Profesores

Administra, registra y actualiza la información de los docentes del sistema.

Información del Profesor

Nombre

Juan

Apellido

Perez

Cédula

1234567890

Especialidad

software

Guardar

Actualizar

Eliminar

Leer Datos

Limpiar

Lista de Profesores Registrados

Buscar profesor...

ID	NOMBRE	APELLIDO	CÉDULA	ESPECIALIDAD
Tabla sin contenido				

Éxito

Profesor registrado correctamente.
Su correo y usuario institucional han sido autogenerados.

Aceptar

Figura 15. Registro de información

Descripción: Formulario utilizado para ingresar nuevos registros al sistema, validando previamente la información antes de almacenarla en la base de datos.

[illegible]

Figura 16. Consulta de registros

Descripción: Pantalla que muestra la información almacenada en la base de datos mediante una tabla (*TableView*), permitiendo visualizar los registros de forma organizada.

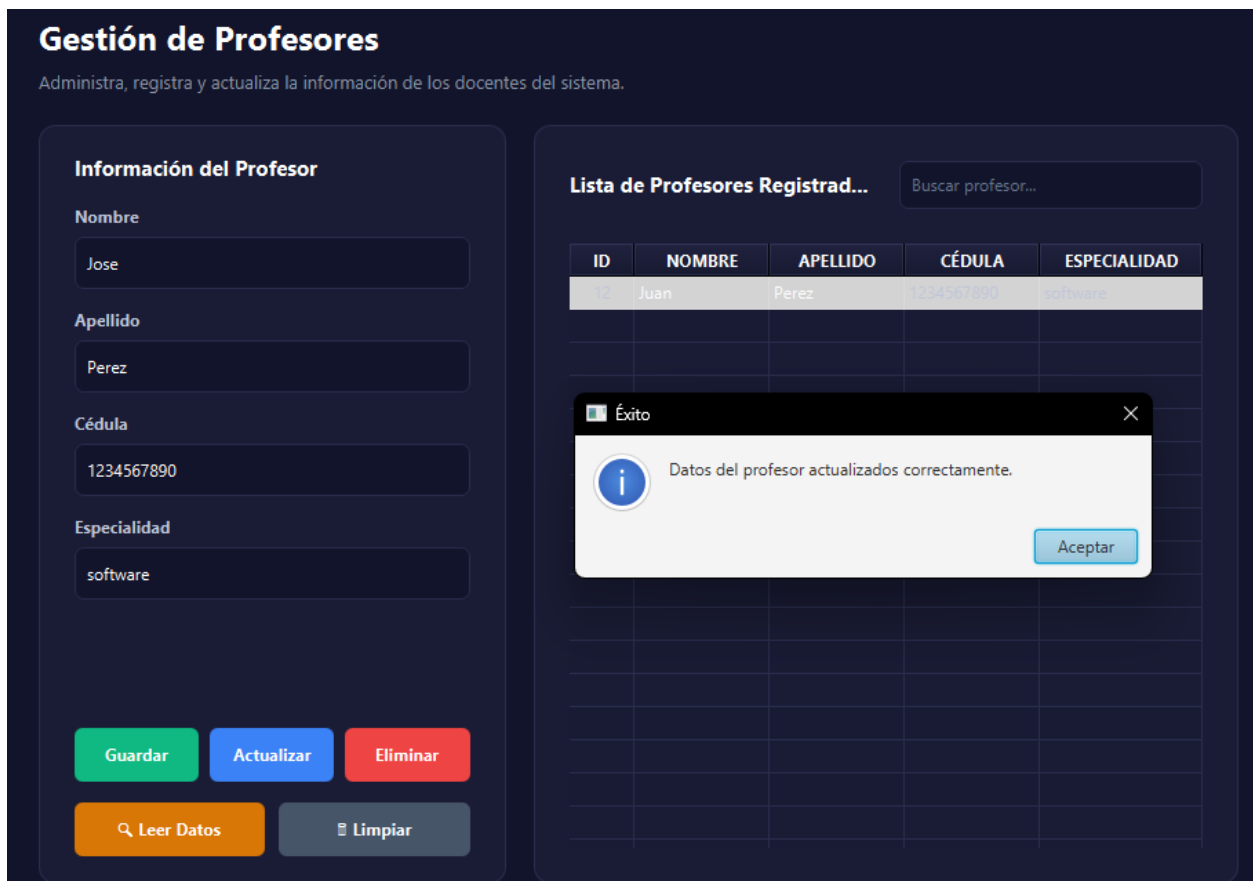


Figura 17. Edición de información

Descripción: Interfaz que permite modificar los datos de un registro existente, actualizando la información almacenada en la base de datos.

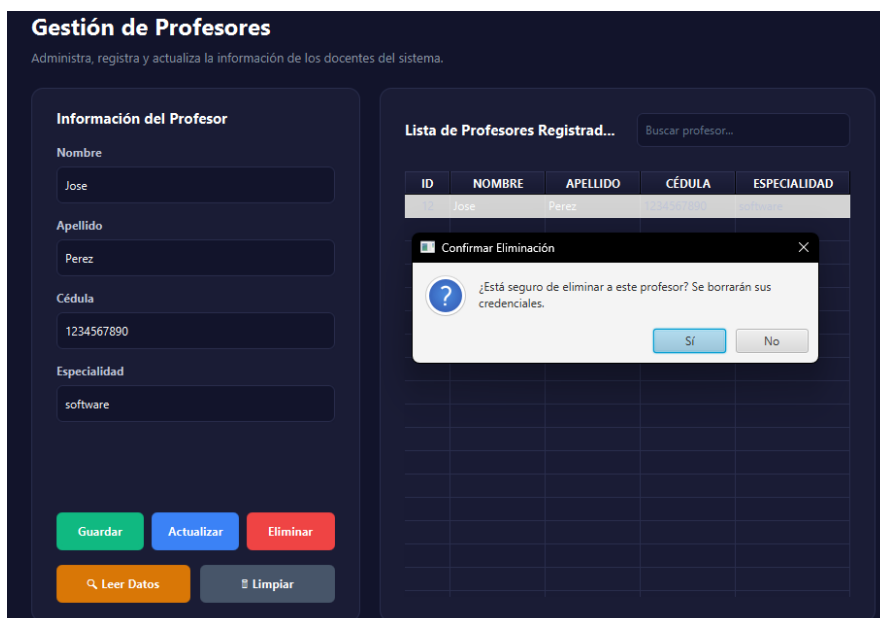


Figura 18. Eliminación de registros

Descripción: Proceso mediante el cual el usuario elimina un registro del sistema, previa confirmación para evitar la eliminación accidental de información.

The screenshot shows a web application interface with a dark theme. On the left, there is a form titled 'Información del Profesor' with fields for 'Nombre' (containing 'Pepe'), 'Apellido' (placeholder 'Ingrese el apellido'), 'Cédula' (containing '15151515'), and 'Especialidad' (containing 'redes'). Below the form are buttons: 'Guardar' (green), 'Actualizar' (blue), 'Eliminar' (red), 'Leer Datos' (orange with a magnifying glass icon), and 'Limpiar' (grey with a trash icon). On the right, there is a section titled 'Lista de Profesores Registrad...' with a search bar 'Buscar profesor...'. Below the search bar is a table with headers: 'ID', 'NOMBRE', 'APELLIDO', 'CÉDULA', and 'ESPECIALIDAD'. The table is currently empty, showing 'Tabla sin contenido'. A modal dialog titled 'Campos Incompletos' is open in the center, featuring a yellow warning triangle icon and the text 'Debe rellenar todos los campos del formulario.' with an 'Aceptar' button.

Figura 19. Validaciones

Descripción: Ejemplo de las validaciones implementadas en el sistema para verificar que la información ingresada sea correcta y completa antes de ejecutar una operación.

The screenshot shows a web application interface with a dark theme. On the left, there is a sidebar titled 'Gestión Educativa' with a 'Reportes' link highlighted in a purple box. The main area is titled 'Gestión de Reportes' with a subtitle 'Genera reportes académicos basados en tus asignaciones actuales.' Below the title is a form titled 'Detalles del Reporte' with fields for 'Título' (placeholder 'Título del reporte'), 'Descripción' (a large text area), 'Fecha' (a date picker), and 'Estado del Reporte' (a dropdown menu). Below the form are buttons: 'Guardar' (green), 'Actualizar' (blue), 'Eliminar' (red), 'Leer' (orange with a magnifying glass icon), and 'Limpiar' (grey with a trash icon). On the right, there is a section titled 'Estudiantes Asignados' with a table that has headers: 'ID A...', 'CARRERA', 'NOMBRE', 'APELLIDO', and 'CORREO'. The table is currently empty, showing 'Tabla sin contenido'.

Figura 20. Reportes

Descripción: Módulo que presenta los reportes generados por el sistema a partir de la información almacenada en la base de datos, facilitando la consulta y el análisis de los registros.

7. Conclusiones

1. El desarrollo del proyecto permitió aplicar de forma práctica los principios de la Programación Orientada a Objetos, utilizando clases, interfaces, encapsulamiento, herencia y polimorfismo para construir una aplicación organizada y mantenible.
2. La integración de JavaFX con MySQL mediante JDBC hizo posible desarrollar un sistema funcional capaz de realizar operaciones CRUD sobre una base de datos real, garantizando la persistencia y consistencia de la información.
3. La implementación de un sistema de autenticación con contraseñas cifradas mediante BCrypt y control de acceso por roles mejoró la seguridad de la aplicación, permitiendo que cada usuario acceda únicamente a las funcionalidades autorizadas.

8. Recomendaciones

1. Incorporar nuevas funcionalidades, como generación de reportes en PDF o Excel, para ampliar las capacidades del sistema y facilitar la administración de la información.
2. Implementar mecanismos de respaldo automático de la base de datos y registros de auditoría para aumentar la seguridad y la recuperación de la información.
3. Continuar aplicando buenas prácticas de desarrollo, como pruebas unitarias, documentación del código y uso constante de control de versiones con GitHub, para facilitar el mantenimiento y evolución del proyecto.
- 4.

9. Bibliografía

- Oracle. *JavaFX Documentation*. <https://openjfx.io/>
- Oracle. *Java Platform, Standard Edition Documentation*. <https://docs.oracle.com/javase/>
- Oracle. *JDBC Basics*. <https://docs.oracle.com/javase/tutorial/jdbc/>
- MySQL. *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/>
- Oracle. *Java Object-Oriented Programming Concepts*. <https://docs.oracle.com/javase/tutorial/java/concepts/>
- OpenJFX Community. *JavaFX API Documentation*. <https://openjfx.io/javadoc/>

10. Anexos

- **Script de Base de Datos**



script_bd_proyecto_
poo.sql

database/script_bd_proyecto_poo.sql

Este archivo contiene la creación de la base de datos, tablas, relaciones y datos de prueba utilizados por el sistema.

- **Repositorio GitHub**

<https://github.com/Alepillajof/Proyecto-POO.git>

- **Ejecutable**



ProyectoFinalPOO.e
xe

dist/ProyectoFinalPOO.exe

- **Video Demostrativo**

<https://youtu.be/U7PqeLC1Tpg?feature=shared>

- **Guia De Usuario**



Manual de
Usuario.pdf

docs/ ManualUsuario_Pillajo_Lacano.pdf